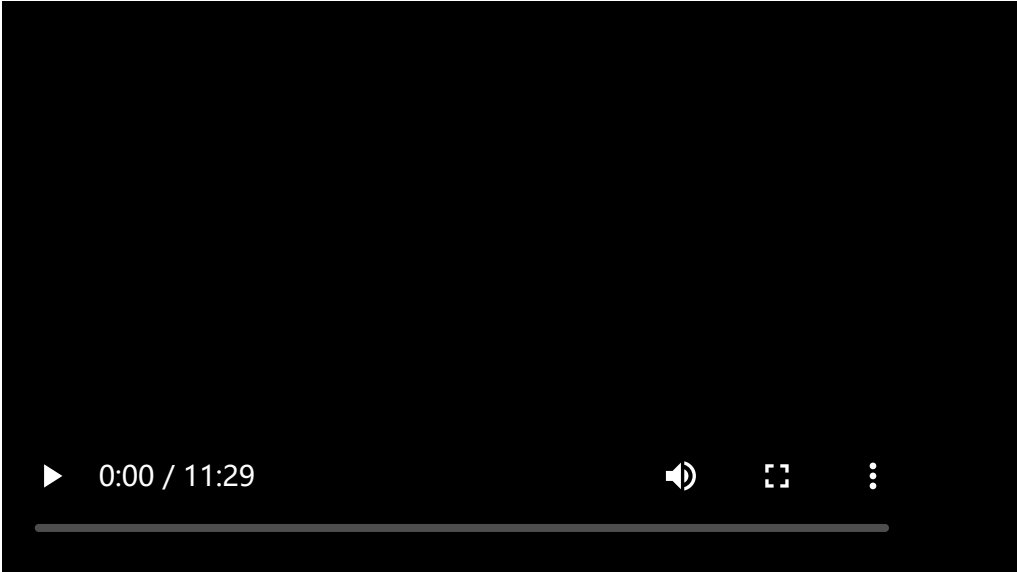


项目简介



任务目标

路径规划 - 生存博弈

鲁班七号在规定时间内（可配置/默认1000步）内，躲避怪物（2个）追击，同时尽可能多的收集宝箱。

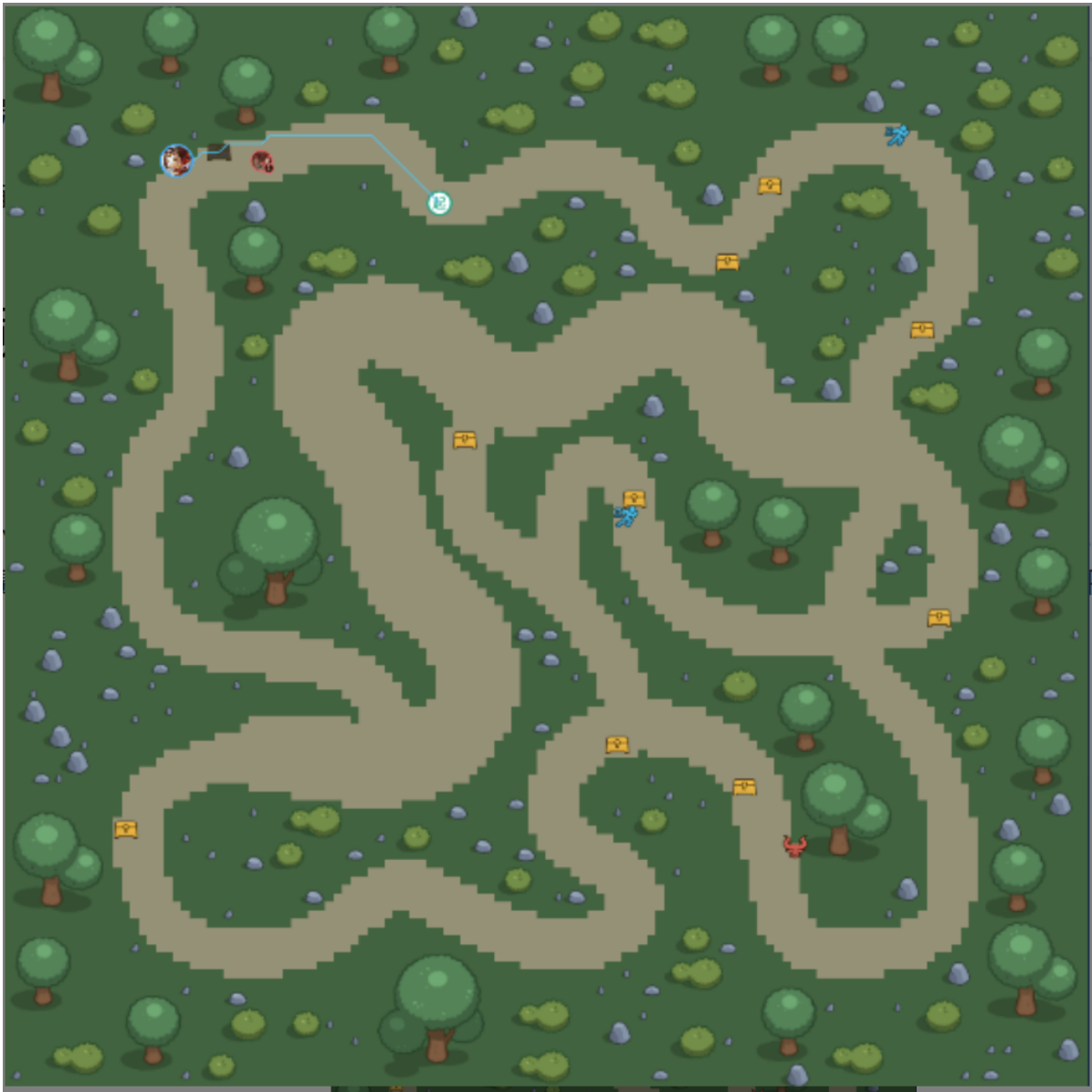
鲁班七号有超级闪现技能，使用可穿越 10或8（斜向）格地形。

地图上有加速增益，拾取后可以增加鲁班七号移速（一步两格）。

环境介绍

地图

地图大小为**128×128（栅格化地图）**，左上角为地图原点 (0,0)，x 轴向右为正，z 轴向下为正。为考察模型泛化能力，本赛题共提供十五张地图：十张开放给选手训练、评估，五张作为隐藏地图用于最终测评。



元素介绍

地图中包含**英雄、怪物、英雄出生点、怪物出生点、宝箱、加速buff、道路、障碍物**。

元素	说明
英雄	环境中存在英雄单位，智能体可以控制英雄在地图中进行移动。本环境采用鲁班七号作为英雄角色，英雄每执行一次动作指令代表一步。
怪物	环境中存在怪物，每一局任务最多生成2个怪物。任务开始时，第一个怪物会从怪物1出生点出发，追逐智能体。当任务进行一定时间后，第二个怪物会从怪物2出生点出发，追逐智能体。当智能体碰到怪物时（英雄距离怪物的距离小于等于1格即为碰到怪物，即英雄出现在以怪物为中心3×3的范围内），任务失败，智能体不可攻击怪物。怪物在默认500步（可配置）后会加速（一步两格）。
英雄出生点	任务开始时，英雄起始位置，可随机出现在非宝箱、加速buf以及障碍物位置的任意道路上。
怪物1出生点	怪物1起始位置，第一只怪物出现时间为任务开始时。怪物1出生点为随机出现在英雄视野域边缘且为道路的位置，即英雄周围21×21的正方形范围的最边缘一圈的道路上。

元素	说明
怪物2出生点	怪物2起始位置，第二只怪物出现时间默认300步（可配置），怪物2出生点为智能体10步前所在位置。
宝箱	智能体可以通过控制英雄拾取宝箱增加积分，每个宝箱获得100积分，地图中最多可配置10个宝箱，宝箱会根据配置数量，随机在地图道路中生成。开发者可以通过配置宝箱的数量来调整环境的复杂度。
加速buff	道路中最大可配置2个加速增益，英雄可以通过拾取加速增益来提升自身的移动速度（一步两格）。拾取后加速效果维持50步，拾取后默认200步后（可配置）刷新。在增益结束后，英雄恢复默认速度。加速增益冷却后会重新刷新。
道路	智能体可以正常移动的区域。在道路中智能体有8个移动方向，智能体在执行一个动作时，会朝该动作的方向移动1步，然后再等待下一次动作指令。
障碍物	障碍物会阻碍智能体的移动，当智能体移动速度为1时向有障碍物的方向移动，将会停留在原地；当智能体移动速度大于1时，如前进方向上存在障碍物，移动至该方向上最后一格无障碍物的位置。

在创建训练任务和评估任务时，上述元素的配置方式有所不同。具体请查看[开发指南-环境配置](#)部分。

英雄

本环境使用**鲁班七号**作为智能体控制的英雄。

属性	说明
数量	1
默认动作空间	16（八个方向移动/八个方向闪现）
视野域	以智能体为中心，分别向上、下、左、右四个方向拓宽10格的正方形区域，即以英雄为中心的21×21的正方形区域
移动速度	1格/步

召唤师技能

技能	说明
闪现	当闪现的目标格子为障碍物时，退回闪现路径上非障碍物的最后一格。当往上、下、左、右闪现时，智能体闪现10格，斜向闪现时，闪现8格。冷却时间100步（可配置）。

计分规则

每局任务开始前，用户可以设定最大步数，英雄在最大步数内（包括最大步数）尽可能存活更多时间，并按下方规则计算任务得分：

任务得分 = 步数得分 + 宝箱得分

- **步数得分：** 任务完成步数 × 奖励系数1.5，任务完成步数为智能体被怪物追到时所使用的步数。
- **宝箱得分：** 每获得一个宝箱，即可增加100分。

环境详述

在开始开发前，请仔细阅读[腾讯开悟强化学习开发框架](#)，深入理解环境、智能体、工作流等核心概念及其相关接口的使用方法。

环境配置

在智能体和环境的交互中，首先会调用 `env.reset` 方法，该方法接受一个 `usr_conf` 参数，这个参数通过读取 ``agent_算法名/conf/train_env_conf.toml`` 文件的内容来实现定制化的环境配置。因此，用户可以通过修改 `train_env_conf.toml` 文件中的内容来调整环境配置。

```
1 | `# usr_conf为代码包中的train_env_conf.toml配置文件，此处以agent_ppo为例`
2 | usr_conf = read_usr_conf("agent_ppo/conf/train_env_conf.toml", logger)
3 | observation, extra_info = env.reset(usr_conf=usr_conf)
```

峡谷追猎环境配置包含的属性如下：

数据名	数据类型	数据描述	取值范围	默认值
map	list of int	训练使用的地图编号列表	[1-10]	[1,2,...,10]
map_random	bool	是否随机抽取地图。true 表示每局从地图列表中随机抽取一张，false 表示按顺序抽取	true/false	false
treasure_count	int	宝箱数量	0-10	10
buff_count	int	加速 buff 数量	0-2	2
buff_cooldown	int	buff 被拾取后重新生成时间（步数）	1-500	200
talent_cooldown	int	闪现技能冷却时间（步数）	50-2000	100
monster_interval	int	第二个怪物出现间隔（步数）。-1表示在可配置范围随机生成对应配置	-1, 11-2000	300
monster_speedup	int	怪物加速增益时间。-1表示在可配置范围随机生成对应配置	-1, 1-2000	500
max_step	int	最大步数，达到时任务结束	1-2000	1000

💡 补充说明：

1. `train_env_conf.toml` 文件中的配置仅在训练时生效，请按上表数据描述进行配置。若配置错误，训练任务会变为"失败"状态，此时可以通过查看env模块的错误日志进行排查。
2. 若需调整模型评估任务时的配置，用户需要通过腾讯开悟平台创建评估任务并完成环境配置，详细参数见[智能体模型评估模式](#)。
3. 智能体出生点会排除宝箱和加速 buff 所在位置。
4. 闪现落点为指定距离范围内最远的可通行格子；若范围内无可通行格子则原地闪现（消耗冷却）。

`train_env_conf.toml` 采用的默认配置：

```
1  [env_conf]
2  # Maps used for training. Customize by keeping only desired map IDs, e.g. [1,
   # 2] for maps 1 and 2.
3  # 训练使用的地图。可自定义选择期望用来训练的地图，如只期望使用1、2号地图训练数组内仅保留
   # [1,2]即可
4  map = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
5
6  # whether to randomly select maps. true = randomly pick one from configured
   # maps per episode, false = used sequentially.
7  # 是否随机抽取地图。布尔值，true表示每局从配置的地图中随机抽取一张，false表示按顺序抽取
   # 地图训练
8  map_random = false
9
10 # Number of treasures. Range: 0~10. in each round, treasure will be randomly
    # generated in accordance with the configure.
11 # 宝箱数量。可配置范围为0~10，每局将按照配置数量在道路上随机生成宝箱。
12 treasure_count = 10
13
14 # Number of speed buffs. Range: 0~2. When set to 1, one of the two buff
    # positions is randomly chosen.
15 # 加速buff数量。可配置范围为0~2。当配置为1时将从每张地图2个buff点位随机选择1个点位生成
    # buff。
16 buff_count = 2
17
18 # Speed buff respawn cooldown in steps after being picked up. Range: 1~500.
19 # 加速buff刷新时间。拾取后重新生成buff时间，单位为步数。可配置范围1~500。
20 buff_cooldown = 200
21
22 # Talent skill cooldown in steps. The skill enters cooldown after being cast.
    # Range: 50~2000.
23 # 技能冷却步数。技能在被施放后会进入冷却状态，冷却状态结束后重新激活。可配置范围为
    # 50~2000。
24 talent_cooldown = 100
25
26 # Steps before the second monster appears. Range: -1, 11~2000. -1 means
    # random within 11~2000.
27 # 第二个怪物出现间隔。即预测多少步后，出现第二个怪物。可配置范围为-1,11~2000。-1即为在11
    # ~2000范围内随机生成对应配置。
28 monster_interval = 300
29
30 # Steps before monster speed increases by one grid. Range: -1, 1~2000. -1
    # means random within 1~2000.
31 # 怪物加速增益时间。即开局多少步后，怪物移动速度增加一格。可配置范围-1,1~2000。-1即为在1
    # ~2000范围内随机生成对应配置。
```

```
32 monster_speedup = 500
33
34 # Maximum steps per episode. The episode ends when this limit is reached.
   Range: 1~2000.
35 # 最大步数。单局任务预测步数达到最大步数时，任务结束。可配置范围为1~2000。
36 max_step = 1000
```

环境信息

视野范围

以智能体为中心，分别向上、下、左、右四个方向拓宽10格的正方形区域，即**21×21**的观测范围。视野域外仅提供怪物的相对位置（相对方向和距离）。

在调用 `env.reset` 与 `env.step` 接口时，会返回环境当前的状态：

```
1 # reset 返回
2 env_obs = env.reset(usr_conf=usr_conf)
3 # step 返回
4 env_reward, env_obs = env.step(hero_actions)
```

`env_reward`包含以下信息：

数据名	数据类型	数据描述
frame_no	int	当前帧号
env_id	string	环境标识
reward	float	当前得分

`env_obs`包含以下信息：

数据名	数据类型	数据描述
env_id	string	环境标识
frame_no	int	当前帧号
observation	Observation	观测信息
extra_info	ExtraInfo	额外信息
terminated	bool	任务是否结束（被怪物捕获）
truncated	bool	任务是否因达到最大步数或异常而截断

下面会对这些数据进行介绍，完整的观测数据结构可以参考[数据协议](#)。

得分信息

`env_reward` 是在当前状态下执行动作 `action` 所获得的分数。

- 1

任务得分 = 步数得分 + 宝箱得分
- 2

步数得分 = 完成步数 × 1.5
- 3

宝箱得分 = 收集宝箱数 × 100

注意：得分用于衡量模型在环境中的表现，也作为衡量强化学习训练产出模型的评价指标，与强化学习里的奖励 `reward` 要区别开。

观测信息 (observation)

数据名	数据类型	数据描述
step_no	int32	当前步数
frame_state	FrameState	帧状态数据
env_info	EnvInfo	环境信息
map_info	int32[][]	局部地图信息（以英雄为中心的视野栅格，1=可通行，0=障碍物）
legal_act	bool[16]	合法动作掩码（16维，true 表示可执行）

额外信息 (extra_info)

数据名	数据类型	数据描述
frame_state	FrameState	全局帧状态数据
map_id	int32	当前地图 ID
result_code	int32	结果码（0=成功，负数=错误）
result_message	string	结果消息

动作空间

共 16 个离散动作：8 个移动动作 + 8 个闪现动作。

动作值	类型	方向	说明
0	移动	右 (East)	向右移动 1 格（有 buff 时移动 1+buff_extra_speed 格）
1	移动	右上 (NE)	向右上移动
2	移动	上 (North)	向上移动
3	移动	左上 (NW)	向左上移动
4	移动	左 (West)	向左移动

动作值	类型	方向	说明
5	移动	左下 (SW)	向左下移动
6	移动	下 (South)	向下移动
7	移动	右下 (SE)	向右下移动
8	闪现	右	向右闪现 10 格
9	闪现	右上	向右上闪现 8 格
10	闪现	上	向上闪现 10 格
11	闪现	左上	向左上闪现 8 格
12	闪现	左	向左闪现 10 格
13	闪现	左下	向左下闪现 8 格
14	闪现	下	向下闪现 10 格
15	闪现	右下	向右下闪现 8 格

方向枚举

1	上 (2)
2	左上 (3) 右上 (1)
3	\ /
4	左 (4)-----+-----右 (0)
5	/ \
6	左下 (5) 右下 (7)
7	下 (6)

合法动作

16 维布尔数组，表示当前帧各动作是否可执行：

索引	动作类型	说明
0-7	移动	始终为 true（移动动作总是可用）
8-15	闪现	取决于闪现技能冷却是否结束且技能未被禁用

执行逻辑

移动规则：

- 普通移动**：每步移动 **1 格**。
- 斜向移动**：目标格子可通行，且相邻两条边（水平/垂直方向）**至少有一条可通行**
- 加速移动**：拾取加速 buff 后，每步移动 **1 + buff_extra_speed 格**（默认 1+1=2 格）。

4. **障碍物处理**：速度为 1 时向障碍物方向移动将**停留在原地**；速度大于 1 时逐格检测，移动至该方向上**最后一格无障碍物**的位置。

闪现规则：

1. **闪现距离**：直线方向（上下左右）**10 格**，斜向方向（四个对角）**8 格**。
2. **冷却时间**：**100 步**（可配置）。
3. **障碍物处理**：从最远距离向近处逐格搜索，闪现到范围内**最远的可通行格子**。若全部不可通行则原地闪现（仍消耗冷却）。
4. **路径收集**：闪现路径上经过的宝箱和 buff 会被收集。

环境监控信息

监控面板中包含了env模块，表示**环境指标**数据，详细说明如下。

面板中文名称	面板英文名称	指标名称	说明
得分	Score	total_score	总得分 = step_score + treasure_score
得分	Score	step_score	步数得分 = current_step × 1.5
得分	Score	treasure_score	宝箱得分 = treasures_collected × 100
地图	Map	total_map	训练地图总数
地图	Map	map_random	是否随机选图（1=是, 0=否）
步数	Step	max_step	任务设置的最大步数
步数	Step	finished_steps	任务结束时的步数
闪现	Flash	flash_count	任务结束时使用闪现技能的次数
闪现	Flash	flash_cooldown	任务开始时，设置的闪现冷却时间
加速增益	Buff	total_buff	任务开始时，设置的加速buff个数
加速增益	Buff	collected_buff	任务结束时收集到的加速buff数量
加速增益	Buff	buff_refresh_time	任务开始时，设置的加速buff冷却时间
宝箱	Treasure	total_treasure	任务设置的宝箱个数
宝箱	Treasure	treasures_collected	任务结束时收集到的宝箱个数
怪物速度	Monster_Speed	monster_speed	怪物的移动速度
怪物出现间隔	Monster_Interval	monster_interval	怪物出现间隔

智能体详述

我们在代码包中提供了智能体的**基础实现**，本文将对该部分内容进行讲解，包括特征处理和奖励处理等。

 **说明：**代码包提供的是一个**最简基线版本**，仅包含基础生存能力。选手需要根据自己对环境的理解，自行扩展特征工程、奖励设计、模型结构等，不断提升智能体的能力。

观测处理

环境返回的 `observation` 信息包含了针对智能体的局部观测信息，可以在 `observation_process` 函数中对这些局部观测信息进行处理。

我们推荐用户使用预处理器 `preprocessor` 对环境返回的 `observation` 信息进行预处理：

```
1 def observation_process(self, env_obs):
2     feature, legal_action, reward = self.preprocessor.feature_process(env_obs,
3     self.last_action)
4     obs_data = ObsData(
5         feature=list(feature),
6         legal_action=legal_action,
7     )
8     remain_info = {"reward": reward}
9     return obs_data, remain_info
```

特征处理

当前基线版本提供了 **40 维**的简化特征向量，布局如下：

部分	维度	含义
hero_self	4	英雄自身状态（pos_x归一化, pos_z归一化, 闪现冷却归一化, 加速buff剩余归一化）
monster_1	5	第一只怪物（is_in_view, pos_x归一化, pos_z归一化, 速度归一化, 距离归一化）
monster_2	5	第二只怪物（同上，未生成时全为0）
map_local	16	以英雄为中心的 4×4 局部地图通行性（从21×21 map_info中心裁取）
legal_action	8	移动方向合法掩码（8维，0-7方向）
progress	2	进度特征（步数归一化, 已存活比例）
合计	40	—

奖励处理

当前基线版本提供了**最简单的生存奖励**设计：

奖励项	含义	数值
survive_reward	每步基础生存奖励	+0.01
dist_shaping	怪物距离塑形（远离怪物有正奖励）	$\pm 0.1 \times \Delta \text{dist}$

最终奖励：`reward = survive_reward + dist_shaping`

时序处理：使用 GAE（Config.GAMMA, Config.LAMDA）计算 advantage 与 reward_sum，作为 PPO 的训练目标。

代码包仅提供了最基础的智能体实现，用户可以仔细阅读[环境详述](#)和[数据协议](#)，根据自己对环境的理解，进行特征工程、奖励开发等工作，不断提升智能体的能力。

算法介绍

我们在代码包中提供了**基础 PPO 算法**实现，同时还提供了一个**diy**模板算法文件夹，用户可在该文件夹中自定义算法实现。

算法	说明
PPO	Proximal Policy Optimization，近端策略优化算法，训练稳定性高，收敛速度快
DIY	用户自定义算法实现模板

算法详细说明

模型设计（Actor-Critic）：

当前基线版本使用**单 MLP 骨干 + 双头**结构：

1	输入（40D）→ MLP骨干（40→128→64）→ Actor头（64→8）→ 动作logits
2	→ Critic头（64→1）→ 状态价值

选手可以自行设计更复杂的网络结构

动作输出：

- 当前基线版本的动作空间为 **8 维离散动作**（仅移动方向 0-7）。
- 合法动作掩码（legal_action）：使用 mask 把非法选项从概率分布中排除（实现上为对 logits 做大负数惩罚再 softmax）。
- 采样策略：训练时随机采样（multinomial），评估时选最大概率（argmax）。

训练流程：

1. 交互采样：Agent 与环境交互生成采样（SampleData），包含 obs, action, prob, value, reward 等。
2. 后处理：填充 next_value 并用 GAE 计算 advantage 与 reward_sum。

- 3. 更新：在采样数据上执行 PPO 更新（策略损失、价值损失、熵正则）。
- 4. 保存/评估：定期保存模型并在验证地图上评估性能。

损失函数：

```
1 total_loss = vf_coef × value_loss + policy_loss - beta × entropy_loss
```

- `value_loss`：Clipped 价值函数损失
- `policy_loss`：PPO Clipped surrogate 目标
- `entropy_loss`：动作熵正则化（鼓励探索）

算法监控信息

算法上报了 reward 等指标，用户可以通过腾讯开悟平台/客户端的监控功能查看。

针对当前基线算法的指标说明如下：

指标名称	说明
total_loss	总损失
policy_loss	策略损失
value_loss	价值损失
entropy_loss	熵损失
reward	累计回报

选手可以 `monitor_builder.py` 中自行添加更多监控指标。

模型保存限制策略

为了避免用户保存模型的频率过于频繁，开悟平台对模型保存会有安全限制，不同的任务会有不同的限制。

默认提供合理的模型保存代码：每30min保存一个模型。支持用户自行实现模型保存的代码，并且能正常按照用户的代码实现保存模型。

模型评估

用户可以在腾讯开悟平台上创建模型评估任务。

地图配置

为考察模型的泛化能力，本赛题共提供 15 张地图：

地图类型	数量	说明
开放地图	10 张（编号 1 ~ 10）	开放给选手进行训练和评估

地图类型	数量	说明
隐藏地图	5 张	用于最终测评，选手不可见

创建评估任务时，可通过平台界面**勾选**需要评估的地图（支持多选 1 ~ 10）

 **泛化性建议：**

- 训练时建议使用**多张地图**进行训练，避免模型过拟合到单一地图
- 评估时建议在**多张不同地图**上测试模型表现，确保策略具备跨地图适应性
- 最终测评将在隐藏地图上进行，模型需要具备对**未见过地图**的泛化能力

环境配置

另外在创建评估任务时，还需对该任务的环境进行配置：

```
1 [env_conf]
2 # 宝箱数量。可配置范围为0~10
3 treasure_count: 10
4
5 # 加速buff数量。可配置范围为0~2
6 buff_count: 2
7
8 # 加速buff刷新时间。单位为步数。可配置范围1~500
9 buff_cooldown: 200
10
11 # 技能冷却步数。可配置范围为50~2000
12 talent_cooldown: 100
13
14 # 第二个怪物出现间隔。可配置范围为-1,11~2000。-1即为在11~2000范围内随机生成
15 monster_interval: 300
16
17 # 怪物加速增益时间。可配置范围-1,1~2000。-1即为在1~2000范围内随机生成
18 monster_speedup: 500
19
20 # 最大步数。可配置范围为1~2000
21 max_step: 1000
```

任务状态：

状态	说明
已完成	与怪物相遇，或达到任务最大步数
异常	各种原因导致的异常

选手开发指引

基线代码包提供了最简单的智能体实现，以下给选手提供部分优化提示，选手们可以自行探索更多优化方向：

1. 特征工程

方向	说明
宝箱特征	添加宝箱位置、距离、优先级等信息
怪物预测	利用历史轨迹预测怪物未来位置
路径规划	评估各方向的逃跑路线质量
地图记忆	记录已探索区域和安全区域
时序特征	利用历史信息辅助决策

2. 奖励设计

方向	说明
宝箱奖励	鼓励收集宝箱提高得分
探索奖励	鼓励探索新区域
闪现奖励	鼓励合理使用闪现逃脱危险
阶段奖励	根据游戏进程调整奖励权重

3. 模型结构

方向	说明
网络加深	增加网络容量以学习复杂策略
注意力机制	动态关注重要特征
多头价值	分解不同目标的价值估计
辅助任务	添加预测任务增强特征学习

4. 动作空间

方向	说明
闪现技能	扩展动作空间以使用闪现（动作8-15）

数据协议

为了方便同学们调用原始数据和特征数据，下面提供了协议供大家查阅。

环境返回数据协议

observation（观测信息）

数据名	数据类型	数据描述
step_no	int32	当前步数
frame_state	FrameState	帧状态数据
env_info	EnvInfo	环境信息
map_info	int32[][]	局部地图信息（以英雄为中心的视野栅格，1=可通行，0=障碍物）
legal_act	bool[16]	合法动作掩码（16维，true 表示可执行）

FrameState（帧状态）

数据名	数据类型	数据描述
frame_no	int32	当前帧号
heroes	HeroState	英雄状态
monsters	MonsterState[]	怪物状态列表
organs	OrganState[]	物件状态列表（宝箱、buff）

HeroState（英雄状态）

数据名	数据类型	数据描述
hero_id	int32	英雄实体 ID
pos	Position	英雄位置 {x, z}（栅格坐标）
treasure_score	float	宝箱得分
step_score	float	步数得分
treasure_collected_count	int32	已收集宝箱数

MonsterState（怪物状态）

数据名	数据类型	数据描述
monster_id	int32	怪物实体 ID

数据名	数据类型	数据描述
pos	Position	怪物当前位置 {x, z}
start_pos	Position	怪物出生位置 {x, z}
hero_l2_distance	int32	与英雄的欧氏距离桶编号 (0-5) 。128×128 地图均匀划分：桶范围：0=[0,30), 1=[30,60), 2=[60,90), 3=[90,120), 4=[120,150), 5=[150,180]
hero_relative_direction	int32	怪物相对于英雄的方位 (0-8)：0=重叠/无效，1=东，2=东北，3=北，4=西北，5=西，6=西南，7=南，8=东南
speed	int32	怪物当前移动速度 (格/步)
monster_interval	int32	第二怪物出现间隔配置值

OrganState (物件状态)

数据名	数据类型	数据描述
sub_type	int32	物件类型 (1=宝箱，2=加速 buff)
config_id	int32	配置 ID (实体 ID)
status	int32	状态 (1=可获取)
pos	Position	物件位置 {x, z}
hero_l2_distance	int32	与英雄的欧氏距离桶编号 (0-5)
hero_relative_direction	int32	物件相对于英雄的方位 (0-8)

EnvInfo (环境信息)

数据名	数据类型	数据描述
total_score	float	总得分
step_no	int32	当前步数
step_score	float	步数得分
pos	Position	英雄位置 {x, z}
treasure_score	float	宝箱得分
treasure_id	int32[]	剩余未收集宝箱 ID 列表
monster_interval	int32	第二怪物出现间隔
total_map	int32	训练地图总数
map_random	int32	是否随机选图 (1=是, 0=否)

数据名	数据类型	数据描述
max_step	int32	最大步数
finished_steps	int32	当前已完成步数
flash_count	int32	闪现技能已使用次数
flash_cooldown	int32	闪现冷却配置值
total_buff	int32	buff 总数量配置
collected_buff	int32	已获取 buff 次数
buff_refresh_time	int32	buff 刷新时间配置
total_treasure	int32	总宝箱数量
treasures_collected	int32	已收集宝箱数
monster_speed	int32	怪物速度配置值

Position（位置）

数据名	数据类型	数据描述
x	int32	X坐标（栅格）
z	int32	Z坐标（栅格）

动作空间协议

动作值	类型	方向	说明
0	移动	右（East）	向右移动 1 格
1	移动	右上（NE）	向右上移动
2	移动	上（North）	向上移动
3	移动	左上（NW）	向左上移动
4	移动	左（West）	向左移动
5	移动	左下（SW）	向左下移动
6	移动	下（South）	向下移动
7	移动	右下（SE）	向右下移动
8	闪现	右	向右闪现 10 格
9	闪现	右上	向右上闪现 8 格
10	闪现	上	向上闪现 10 格
11	闪现	左上	向左上闪现 8 格

动作值	类型	方向	说明
12	闪现	左	向左闪现 10 格
13	闪现	左下	向左下闪现 8 格
14	闪现	下	向下闪现 10 格
15	闪现	右下	向右下闪现 8 格

方向枚举协议

值	方向
0	重叠/无效
1	东
2	东北
3	北
4	西北
5	西
6	西南
7	南
8	东南

任务状态协议

状态	说明
running	任务进行中
completed	任务完成（达到最大步数且未被抓住）
failed	任务失败（被怪物抓住）

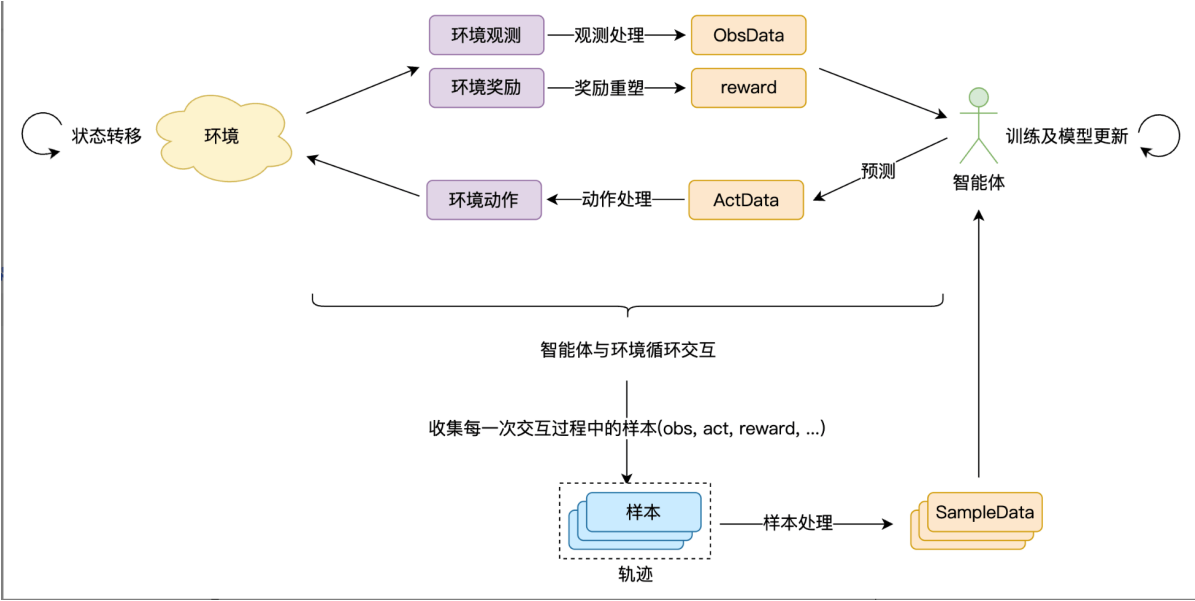
综述

欢迎来到腾讯开悟！

腾讯开悟强化学习开发框架是基于[强化学习系统系列技术标准](#)打造的标准化开发套件。该框架为开发者提供了标准化的编程接口和丰富的工具集，支持开发者高效完成智能体开发、环境交互，以及模型的训练及预测流程。

训练流程简介

本开发框架的完整训练流程如下图所示：



如图，完整训练流程包含以下关键环节：

环节	介绍
智能体-环境循环交互	- 智能体将环境提供的观测和奖励处理为符合预测函数输入要求的数据； - 调用预测函数，生成动作指令； - 将智能体输出的动作指令处理为符合环境输入要求的数据； - 环境执行动作后完成状态转移，并反馈新的观测数据和奖励数据；
样本处理	- 每个环境有不同的开始与结束逻辑，智能体与环境从开始到结束的完整交互过程，称为episode； - 智能体与环境每一次交互产生的结构化数据，称为 样本 ；一个episode产生的样本序列称为 轨迹 ； - 对轨迹数据进行处理，转换为规范化 训练样本(SampleData) ；
模型迭代优化	- 基于训练样本，通过算法持续更新模型参数，实现策略优化；
智能体模型更新	- 智能体加载最新模型，与环境继续循环交互；

该流程通过强化学习分布式计算框架提供的训练工作流实现。基于此，开发框架主要包含三大核心模块：

- [强化学习环境系统](#)：提供标准的**强化学习环境接口**。开发者可以通过标准接口，实现智能体与环境的交互。
- [强化学习智能体开发套件](#)：提供标准的**强化学习智能体接口**，以及算法库、模型组件库等工具函数库。开发者可以通过工具函数库快速完成智能体的构建。
- [强化学习分布式计算框架](#)：提供标准接口，支持开发者按需实现训练工作流，运行单机或分布式的训练及评估任务。

代码包简介

开发者可以通过腾讯开悟平台所提供的强化学习项目使用开发框架。一个强化学习项目的代码目录如下：

```
1  📁 根目录
2  ├── 📁 agent
3  │   ├── 📁 algorithm
4  │   │   ├── __init__.py
5  │   │   └── algorithm.py
6  │   ├── 📁 conf
7  │   │   ├── __init__.py
8  │   │   ├── conf.py
9  │   │   └── train_env_conf.toml
10  │   ├── 📁 feature
11  │   │   ├── __init__.py
12  │   │   ├── definition.py
13  │   │   └── preprocessor.py
14  │   ├── 📁 model
15  │   │   ├── __init__.py
16  │   │   └── model.py
17  │   ├── 📁 workflow
18  │   │   ├── __init__.py
19  │   │   └── train_workflow.py
20  │   ├── __init__.py
21  │   └── agent.py
22  ├── 📁 conf
23  │   ├── __init__.py
24  │   └── configure_app.toml
25  ├── 📁 log
26  └── train_test.py
```

代码目录介绍：

目录名	介绍
agent/	智能体子目录，智能体相关内容均集中于该目录，是开发者核心工作目录。
conf/	配置文件目录，包含运行训练任务相关的配置，例如训练样本批处理大小 batch_size等。
log/	日志目录，存放运行代码测试脚本时生成的日志文件。
train_test.py	代码正确性测试脚本，该脚本会使用当前代码包完成一步训练。建议开发者在启动训练任务前，确保代码已通过该脚本检测。

agent

目录/文件名	介绍
algorithm/	算法相关，开发者在该目录下完成算法实现，包含loss计算、模型优化等，详情见 算法开发
feature/	特征相关，开发者在该目录下完成数据结构定义和数据处理方法，以及样本处理和奖励计算，详情见 数据处理与奖励设计
model/	模型相关，开发者在该目录下完成模型实现。详情见 模型开发
workflow/	工作流目录，开发者在该目录下完成训练工作流的开发。详情见 工作流开发
agent.py	智能体核心代码文件，开发者在该文件中完成预测、训练等核心函数的实现。详情见 智能体开发

标准代码包中都存在一个agent_diy子文件夹，该文件夹是预定义的智能体模板，可供开发者进行智能体的开发。

conf

文件名	介绍
configure_app.toml	训练任务相关的配置，包括样本大小、样本池大小等。

通过对训练流程和代码包的介绍，相信开发者能够对腾讯开悟开发框架建立了初步认知。

接下来，我们将详细介绍每个模块的功能及使用方式。

环境

在综述中提到，强化学习训练流程离不开智能体与环境的持续交互，本文将详细介绍强化学习环境系统的功能及标准接口函数。

概述

强化学习环境是基于输入动作，输出观测、奖励等反馈的功能模块，用于表达强化学习算法所求解的问题场景。

开发框架通过场景适配模块，对仿真器进行封装，将其特化的接口、协议转换为强化学习环境统一的接口和协议，供智能体调用。

强化学习环境系统主要提供如下功能：

1. 接收配置信息，用于指定自身初始化方式，比如环境中各种元素的初始状态。
2. 输出观测、奖励信息，可用于智能体预测、训练。
3. 输出观测、奖励之外的其他信息，供强化学习系统相关组件使用以实现特定功能。其他信息可包括可视化数据、日志数据等，实现的功能包括环境可视化、运行状况监测等。
4. 接收动作指令，完成状态转移并产生新的观测和奖励。

环境使用

开发框架通过场景适配模块，将问题场景进行标准化封装，为开发者提供统一的交互接口与通信协议。由于环境之间存在差异，接口中所涉及的观测、奖励等信息的具体数据结构也有所不同，开发者需查阅所使用环境的官方数据协议文档以获取准确信息。

开发者可以在[训练工作流](#)的workflow中获取到对应环境的实例，通过标准接口实现智能体与环境的交互。

核心函数介绍

reset(usr_conf)

reset会将环境重置为环境配置文件中指定的状态，并且返回初始观测。

```
1 # usr_conf为开发者传入的环境配置
2 obs, state = env.reset(usr_conf = usr_conf)
```

Parameters

参数名	介绍
usr_conf	dict类型，环境配置文件

Returns

参数名	介绍
obs	dict类型，环境观测信息
state	dict类型，环境全局信息

step(act, stop_game = false)

环境会执行传入的act动作指令，完成一次状态转移，并返回新的观测和奖励等信息。

```
1 frame_no, _obs, score, terminated, truncated, _state = env.step(act, stop_game = false)
```

Parameters

参数名	介绍
act	dict类型，环境执行的动作
stop_game	bool类型，是否结束当前对局

Returns

参数名	介绍
frame_no	int类型，当前环境实例运行时的帧号
_obs	dict字典类型，当前帧的观测信息
score	int类型，当前帧的奖励信息
terminated	bool类型，当前环境实例是否结束
truncated	bool类型，当前环境实例是否异常或中断
_state	dict字典类型，当前帧的全部状态信息

综述

智能体是强化学习系统中的核心模块，在[开发框架综述](#)中提到，完整训练流程包括：

环节	介绍
智能体-环境循环交互	- 智能体将环境提供的观测和奖励处理为符合预测函数输入要求的数据； - 调用预测函数，生成动作指令； - 将智能体输出的动作指令处理为符合环境输入要求的数据； - 环境执行动作后完成状态转移，并反馈新的观测数据和奖励数据；
样本处理	- 每个环境有不同的开始与结束逻辑，智能体与环境从开始到结束的完整交互过程，称为episode； - 智能体与环境每一次交互产生的结构化数据，称为 样本 ；一个episode产生的样本序列称为 轨迹 ； - 对轨迹数据进行处理，转换为规范化 训练样本 (SampleData) ；
模型迭代优化	- 基于训练样本，通过算法持续更新模型参数，实现策略优化；
智能体模型更新	- 智能体加载最新模型，与环境继续循环交互；

基于上述训练流程，我们将智能体的开发分为四个部分：

1. [数据处理及奖励设计](#)：介绍基于环境观测数据进行特征处理、样本处理和奖励设计的方法。
2. [模型开发](#)：介绍模型开发接口及开发方法。
3. [算法开发](#)：介绍包括算法开发接口及开发方法。
4. [工作流开发](#)：介绍开发者开发自定义训练工作流的方法。

接下来，将通过独立的章节对强化学习智能体开发套件中每个模块的功能及接口函数进行介绍。

特征处理

环境返回的数据通常无法直接作为智能体预测和训练的输入，开发者需要完成特征处理、样本处理和奖励设计，确保数据结构与类型符合智能体的接口规范。

特征处理

在特征处理时，开发者需要完成四个关键的开发工作，分别是**定义数据结构**、**观测处理**、**动作处理**。

定义数据结构

开发目录：<智能体文件夹>/feature/definition.py

首先，开发者需要定义智能体可以使用的数据结构（类）。

开发框架已经预先定义好了三种数据类型：ObsData, ActData, SampleData。

- ObsData和ActData分别表示智能体预测的输入和输出，将会由 agent.predict() 使用；
- SampleData为训练样本的数据类型，训练样本将会被 agent.learn() 使用，进行模型训练。

核心函数介绍

create_cls

用于动态创建数据结构（类）。ObsData, ActData, SampleData是训练流程必需的三类，但每一个类的数据结构包含哪些属性完全由开发者自定义，属性名称和属性数量没有限制。

```
1  ObsData = create_cls("ObsData",
2      feature=None,
3  )
4  ActData = create_cls("ActData",
5      action=None,
6      prob=None,
7  )
8  SampleData = create_cls("SampleData",
9      npdata=None
10 )
```

Parameters

参数名	介绍
第一个参数	字符串类型，类的名称
其余参数	类的属性，默认值为None，由开发者自行定义

观测处理

开发目录：<智能体文件夹>/agent.py

由于环境的reset和step接口返回的数据属于原始观测数据，无法直接作为智能体预测时的输入，开发者需要将这部分数据进行特征化。

核心函数介绍

observation_process

将环境返回的观测数据转换成ObsData类型数据。
很多情况下，特征工程包含了大量的数值处理、数据转换和领域知识，我们建议将大量的特征处理代码在 <智能体文件夹>/feature/preprocessor.py 文件中实现，然后由于 observation_process 进行调用。

```
1 def observation_process(self, obs, state=None):
2     return ObsData(feature=feature, legal_act=legal_actions)
```

Parameters

参数名	介绍
obs	Observation类型，env.reset和env.step返回的环境观测数据
state	EnvInfo类型，env.reset和env.step返回的环境状态数据

Return

参数名	介绍
ObsData	开发者定义的ObsData类型的数据，将作为 agent.predict() 函数的输入。

动作处理

开发目录：<智能体文件夹>/agent.py

由于环境的step接口的输入须要满足环境的特定数据协议，开发者需要将智能体预测的输出转换为符合环境step接口输入要求的数据。

核心函数介绍

action_process

将智能体预测输出的ActData类型数据转换成环境可以接收的动作数据。

```
1 def action_process(self, act_data):
2     return act_data.act
```

Parameters

参数名	介绍
act_data	开发者定义的ActData类型的数据

Return

环境能处理的动作数据类型，作为 `env.step()` 的输入

奖励设计

开发目录：<智能体名称>/feature/definition.py

这里的奖励特指强化学习中的Reward，注意要与环境反馈的Score进行区分。Score通常用于衡量智能体在环境中的实际表现。开发者在设计Reward时，有非常大的灵活性，不仅可以基于环境返回的观测信息，还可以加入开发者对问题的理解、经验或者知识。

核心函数介绍

reward_shaping

开发框架预设的奖励设计函数接口，开发者可以通过该函数实现复杂的奖励计算，在训练工作流程中调用。

```
1 def reward_shaping(obs, _obs, state, _state):
2     return reward
```

Parameters

参数个数和类型不限制，可以是环境信息、智能体信息、开发者的经验和知识等。

Return

数值类型，计算出的reward值

样本处理

开发目录：<智能体文件夹>/feature/definition.py

由于环境与智能体交互过程中产生的轨迹数据无法直接作为智能体训练时的输入，开发者需要将轨迹数据转换为训练样本数据。

核心函数介绍

sample_process

将环境与智能体交互过程中产生的轨迹数据转换成开发者定义的SampleData类型数据。

```
1 @attached
2 def sample_process(self, list_game_data):
3     return [SampleData(**i.__dict__) for i in list_game_data]
```

Parameters

参数名	介绍
list_game_data	list(Frame)类型，使用开发者自定义的Frame作为输入，因为样本一般进行批处理，所以传入列表

Return

参数名	介绍
list(SampleData)类型	SampleData类型的数据组成的列表

为了支持分布式训练，样本数据需要进行网络传输，由于SampleData无法直接进行网络传输，需要先转换成Numpy的Array，待传输到对端之后再由np.Array转换成SampleData。

因此，开发者需要实现两个转换函数 `SampleData2NumpyData` 和 `NumpyData2SampleData`，这两个函数互为反函数。

注意：由于这两个函数会被分布式计算框架调用，因此这两个函数的实现都必须包含一个装饰器 `@attached`

SampleData2NumpyData

将SampleData转换为NumpyData。

```
1 | @attached
2 | def SampleData2NumpyData(g_data):
3 |     return g_data.npdata
```

Parameters

参数名	介绍
g_data	SampleData 类型

Return

Numpy.array类型

NumpyData2SampleData

将NumpyData转换为SampleData。

```
1 | @attached
2 | def NumpyData2SampleData(s_data):
3 |     return SampleData(npdata=s_data)
```

Parameters

参数名	介绍
s_data	Numpy.array 类型

Return type

SampleData类型

算法开发

开发目录: <智能体名称>/algorithm/algorithm.py

在完成特征处理和奖励设计后，开发者还需要实现强化学习算法，以通过特定优化方法更新模型参数。

以下为实现强化学习算法的核心函数介绍，有关函数的更多细节可以查阅[分布式计算框架](#)

核心函数介绍

learn

实现强化学习优化算法的核心方法，该函数输入为训练样本数据，开发者需基于不同的算法完成相关实现，包括优化方法、损失计算等。

```
1 def learn(self, list_sample_data):
2     """
3     Implementing the core method of the algorithm
4     实现算法的核心方法
5     """
6     loss = 0                                # 基于不同算法实现loss计算 Calculate loss
7     loss.backward()                          # 计算梯度 Calculate gradient
8     self.optimizer.step()                   # 通过梯度下降等方法更新模型 Update weights
```

Parameters

参数名	介绍
list_sample_data	list类型，训练样本(SampleData)列表

模型开发

开发目录: <智能体名称>/model/model.py

一个强化学习模型是基于特征作为输入数据，输出策略的神经网络模型。

开发者需要在 model.py 文件中，实现神经网络模型。开发框架要求，模型类需继承 torch.nn.Module 类，即符合Pytorch模型的实现规范。

```
1 class Model(nn.Module):
2     def __init__(self, state_shape, action_shape=0, softmax=False):
3         super().__init__()
```

workflow开发

训练 workflow

在完成智能体开发后，需要进一步实现由[分布式计算框架](#)提供的训练 workflow 接口，使智能体和环境持续交互，收集训练样本，迭代模型参数，最终完成策略的优化。

核心函数介绍

workflow

通过该函数实现强化学习训练 workflow，调用智能体和环境提供的接口，完成环境交互、样本收集和模型更新。

```
1 @attached
2 def workflow(envs, agents, logger=None, monitor=None):
```

Parameters

参数名	介绍
envs	list类型，环境列表，返回当前正在运行的环境。
agents	list类型，智能体列表，通过调用开发者实现的 <code><智能体名称>/agent.py</code> 实例化 Agent, 并作为输入传入 <code>workflow</code> 。
logger	Logger类型，框架提供的日志组件，接口与 python 的 <code>logging</code> 库一致。
monitor	Monitor类型，框架提供的监控组件。

接下来，我们将通过一个训练 workflow 关键步骤的代码示例（具体实现由开发者完成），说明如何通过训练 workflow 实现完整训练流程。

```
1 @attached
2 def workflow(envs, agents, logger=None, monitor=None):
3     # Get the environment and agent
4     # 获取环境和智能体
5     env, agent = envs[0], agents[0]
6
7     # Execute several epochs
8     # 执行若干次 epoch
9     epoch_num = 1000
10
```

```

11 # Each epoch executes several episodes
12 # 每个epoch执行若干个episode
13 episode_num_every_epoch = 1000
14
15 # Training loop
16 # 训练循环
17 for epoch in range(epoch_num):
18     # After each episode, the trajectory data is converted into
training samples for training.
19     # 在每一个episode结束之后，将轨迹数据转换成训练样本进行训练
20     for g_data in run_episodes(episode_num_every_epoch, env, agent,
logger, monitor):
21         # Agent training. If single-machine training, the model is
trained directly; if distributed training, samples are sent to the sample-
pool.
22         # agent进行训练。如果是单机训练，则直接对模型进行训练；如果是分布式训练，则
将训练样本发送到样本池。
23         agent.learn(g_data)
24         # Ensure that the next training sample collected is new
25         # 清空g_data，确保下一次搜集的训练样本是新的
26         g_data.clear()
27
28     # Save the model at intervals
29     # 依据时间间隔保存模型
30     now = time.time()
31     if now - last_save_model_time >= 300:
32         agent.save_model()
33         last_save_model_time = now
34
35
36 def run_episodes(n_episode, env, agent, logger, monitor):
37     # Run several episodes
38     # 运行若干个episode
39     for episode in range(n_episode):
40         # Reset data at the beginning of an episode
41         # 在episode开始时重置数据
42         done = False
43         collector = list()
44
45         # Reset enviroment and get initial info
46         # 重置环境，并获取环境初始状态
47         obs, state = env.reset(usr_conf=usr_conf)
48
49         # Load the latest model and call it on demand; if in stand-alone
mode, there is no need to load the remote model
50         # 加载最新模型，按需调用；若训练采用单机模式，则无需加载远程模型，可不调用该函数
51         agent.load_model(id="latest")
52
53         # Run an episode loop
54         # 运行一个episode循环
55         while not done:
56             # Agent performs inference, gets the predicted action for the
next frame
57             # 调用智能体预测函数，获取下一时刻的动作
58             act_data = agent.predict(list_obs_data=[obs_data])[0]
59

```

```

60         # Unpack ActData into action
61         # 将智能体输出的ActData数据转换为符合环境数据协议要求的动作数据
62         act = agent.action_process(act_data)
63
64         # Interact with the environment, execute actions, get the next
state
65         # 调用环境step接口，与环境交互，执行动作，获取下一时刻的状态
66         frame_no, _obs, score, terminated, truncated, _state =
env.step(act)
67         if _obs == None:
68             break
69
70         # Feature processing
71         # 对环境返回的观测数据进行处理
72         _obs_data = agent.observation_process(_obs, _state)
73
74         # Disaster recovery
75         # 容灾
76         if truncated and frame_no == None:
77             break
78
79         # Calculate reward
80         # 计算reward
81         reward = reward_shaping(obs_data, _obs_data, state, _state)
82
83         # Episode done signal
84         # episode结束信号
85         done = terminated or truncated
86
87         # Construct sample
88         # 构造样本
89         frame = Frame(
90             obs=obs_data.feature,
91             _obs=_obs_data.feature,
92             act=act,
93             rew=reward,
94             done=done,
95         )
96         collector.append(frame)
97
98         # If the game is over, the sample is processed and sent to
training
99         # 如果episode结束，则进行样本处理，将样本送去训练
100         if done:
101             if len(collector) > 0:
102                 collector = sample_process(collector)
103                 # Return samples
104                 # 返回样本数据，agent会调用agent.learn(g_data)进行训练
105                 yield collector
106                 break
107
108         # Status update
109         # 状态更新
110         obs_data = _obs_data
111         obs = _obs
112         state = _state

```

智能体开发

开发目录: `<智能体名称>/agent.py`

在完成模型和算法后，开发者还需要实现强化学习智能体，智能体使用模型进行决策、与环境交互并通过算法更新模型参数。

以下为实现强化学习智能体的核心函数介绍，有关函数的更多细节可以查阅[分布式计算框架](#)

核心函数介绍

learn

该函数输入为训练样本数据，开发者需要在该函数中调用算法消费训练样本进行训练。

当然，在不同的训练模式下，该函数使用方法有所不同：

- 单机训练：开发者需要在训练 workflows 中手动调用该函数以进行进一步训练。
- 分布式训练：
 - 该函数作为训练函数会被循环执行，无需开发者手动调用。
 - 但该函数还作为样本发送函数，开发者需要在训练 workflows 中手动调用，以将样本发送至样本池。

```
1 def learn(self, list_sample_data):
2     self.algo.learn(list_sample_data)           # 调用算法消费训练样本进行训练 Call
    algorithm to train model
```

Parameters

参数名	介绍
list_sample_data	list类型，训练样本(SampleData)列表

predict

该方法通过调用模型进行预测，通常在训练时调用该方法，依策略的概率分布采样或引入随机概率。

```
1 @predict_wrapper
2 def predict(self, list_obs_data, list_state):
3     return [ActData]
```

Parameters

参数名	介绍
list_obs_data	list类型，观测数据(ObsData)列表
list_state	可选参数，list类型，环境返回的状态数据列表

Return

参数名	介绍
List(ActData)	list类型，开发者定义的动作数据(ActData)列表

exploit

该方法通过调用模型进行预测，通常在评估时调用该方法，选取策略中概率最高的动作或者策略认为最优的动作。

```
1 | @exploit_wrapper
2 | def exploit(self, observation):
```

Parameters

参数名	介绍
observation	dict类型，环境观测字典，评估工作流程中将原始的环境观测信息作为输入传入 <code>agent.exploit()</code> 。

Return

参数名	介绍
action	list类型，动作列表，环境可以直接使用的动作指令

load_model

智能体通过该接口完成模型参数加载。在上文中提到，Actor会从模型池中获取最新模型参数文件，开发者需要手动调用 `load_model()` 函数，使智能体完成模型参数加载。

```
1 | @load_model_wrapper
2 | def load_model(self, path=None, id="1"):
3 |     # when loading the model, you can load multiple files,
4 |     # and it is important to ensure that each filename matches the one used
5 |     # 加载模型，可以加载多个文件，注意每个文件名需要和save_model时保持一致
6 |     model_file_path = f"{path}/model.ckpt-{str(id)}.pk1"
7 |     self.model.load_state_dict(
8 |         torch.load(model_file_path, map_location=self.device),
9 |     )
```

Parameters

参数名	介绍
path	string类型，加载模型参数文件的路径，开发框架根据使用场景得到相应的路径, 并作为输入传入 <code>load_model</code>
id	string类型，模型参数文件的 id，使用 id 指定加载的模型参数文件

save_model

开发者可以通过该函数保存当前时刻的模型文件及智能体代码包，开发框架会将开发者需要保存的内容打包为zip格式的文件。

当开发者使用腾讯开悟客户端开发时，开发框架会在客户端指定目录下存储该zip文件。
当开发者使用腾讯开悟平台时，开发框架会将该zip文件存储在云端，开发者可以通过平台的训练管理模块查看每一个训练任务的zip文件，即模型。

```
1 @save_model_wrapper
2 def save_model(self, list_obs_data, list_state):
3     # To save the model, it can consist of multiple files,
4     # and it is important to ensure that each filename includes the
5     # "model.ckpt-id" field.
6     # 保存模型，可以是多个文件，需要确保每个文件名里包括了model.ckpt-id字段
7     model_file_path = f"{path}/model.ckpt-{str(id)}.pkl"
8
9     # Copy the model's state dictionary to the CPU
10    # 将模型的状态字典拷贝到CPU
11    model_state_dict_cpu = {k: v.clone().cpu() for k, v in
self.model.state_dict().items()}
    torch.save(model_state_dict_cpu, model_file_path)
```

Parameters

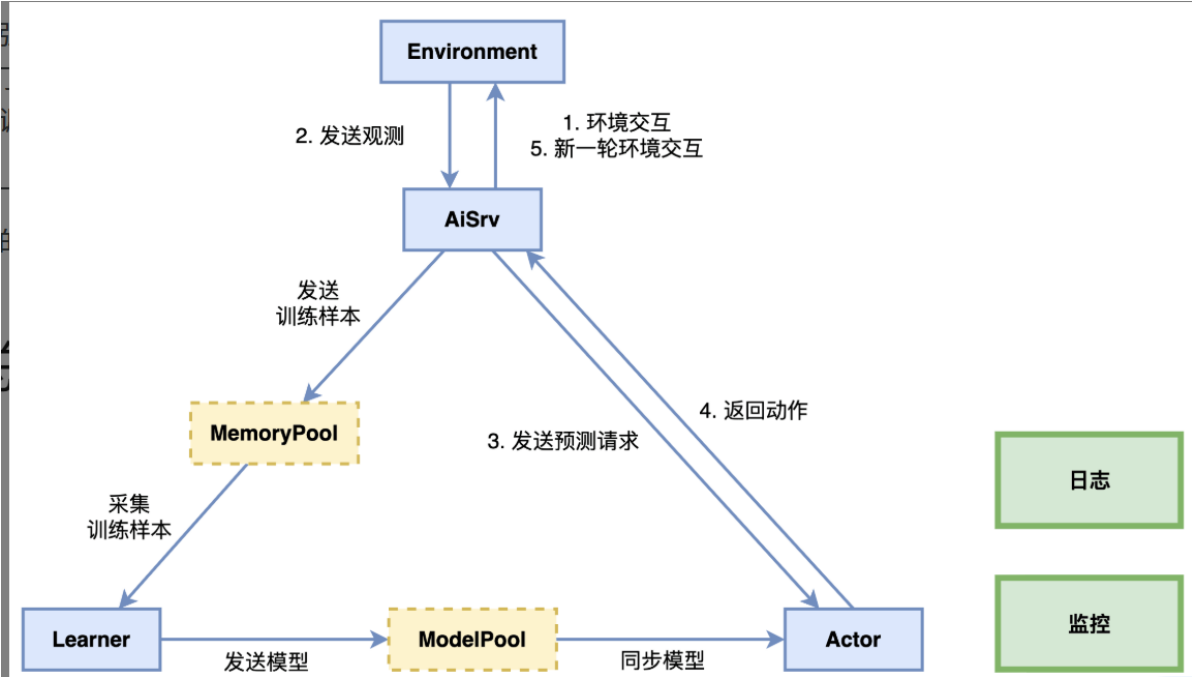
参数名	介绍
path	string类型，模型文件保存的路径，开发框架根据使用场景得到相应的路径, 并作为输入传入 <code>save_model</code>
id	string类型，模型文件的索引，开发框架获取到模型池中最新模型的索引, 并作为输入传入 <code>save_model</code>

分布式计算框架

在强化学习项目的开发中，分布式计算框架是支撑大规模训练任务的核心基础设施。本开发框架提供了由腾讯王者荣耀团队自主研发的强化学习分布式计算框架KaiwuDRL，通过并行化计算、高效资源调度和分布式协同优化，显著提升智能体训练的效率与稳定性。

接下来，我们将详细介绍KaiwuDRL的系统架构与核心能力，帮助开发者进一步理解训练和评估工作流的运行逻辑。

总体架构



组件介绍

如上图所示，KaiwuDRL 的整体架构包括 Environment、Aisrv、Actor、Learner 等强化学习组件（均支持多实例并行运行）。此外，还集成了通信、日志、监控、对象存储等基础组件。

组件介绍如下表：

组件名称	功能描述
Environment	环境服务组件，负责运行强化学习环境，支持通过标准接口与环境交互，并返回环境的观测obs。
Aisrv	训练流程中枢，负责收集环境样本，运行训练、评估工作流，以及处理各个组件间的数据传输。
Actor	预测服务组件，负责响应Aisrv的预测请求，调用智能体 predict() 或 exploit() 函数生成动作决策结果。
Learner	训练服务组件，负责采集训练样本，调用智能体 learn() 函数完成梯度计算及模型迭代。
MemoryPool	样本存储组件，简称样本池。负责存储训练样本，接收 Aisrv 打包的训练样本，发往 Learner 用于智能体训练。

组件名称	功能描述
ModelPool	模型存储组件，简称模型池。负责存储模型参数文件，接收 Learner 产出的模型参数文件，将最新的模型参数文件发送给Actor。
日志	日志采集组件，负责记录强化学习系统中各个组件的运行日志，支持通过标准接口上报日志。
监控	监控采集组件，负责采集系统资源使用率、训练指标趋势等数据，支持通过标准接口上报数据指标。

MemoryPool和ModelPool仅在分布式训练时启用。

服务介绍

基于上述组件，KaiwuDRL 提供了预测服务和训练服务：

预测服务

1. Aisrv → Environment：发送环境配置并创建新一局episode；
2. Environment → Aisrv：返回原始观测数据；
3. Aisrv → Actor：Aisrv基于原始观测数据，向Actor发送预测请求；
4. Actor：使用预测请求中的观测进行特征处理，智能体基于特征处理后的数据进行预测，并且将预测数据处理为环境可以识别的动作指令，发送给Aisrv；
5. Aisrv：使用动作指令与环境Environment进行交互，Environment返回新的观测；

训练服务

1. Aisrv：预测服务不断产生轨迹数据，Aisrv完成样本处理，并发送至样本池；
2. Learner：从样本池按批采集样本进行训练，并将最新的模型参数同步至Actor；

工作流

KaiwuDRL提供了训练、评估工作流的接口函数，开发者可以按需灵活调用上述组件和服务，以实现模型的训练和评估。

训练工作流

开发目录：`<智能体名称>/workflow/train_workflow.py`

workflow

训练工作流的核心函数，在workflow中可自定义训练流程。可以在[智能体/工作流开发](#)中查看详细的训练工作流代码示例。

```
1 @attached
2 def workflow(envs, agents, logger=None, monitor=None):
```

Parameters

参数名	介绍
envs	list类型，环境列表，返回当前正在运行的环境。
agents	list类型，智能体列表，通过调用开发者实现的 <code><智能体名称>/agent.py</code> 实例化 Agent, 并作为输入传入 <code>workflow</code> 。
logger	Logger类型，框架提供的日志组件，接口与 python 的 <code>logging</code> 库一致。
monitor	Monitor类型，框架提供的监控组件。

评估 workflow

在运行训练任务（训练 workflow）并获得模型文件后，可以通过运行评估任务（评估 workflow），对模型能力进行验证。

当开发者在使用腾讯开悟平台所提供的强化学习项目时，评估 workflow 由腾讯开悟官方实现，开发者无法修改。评估 workflow 会调用开发者自定义的 `agent.exploit()` 函数。

Agent

KaiwuDRL 提供了智能体相关的接口函数，开发者可以按需实现以下接口函数，并在训练 workflow 中调用。

开发目录：`<智能体名称>/agent.py`

learn

该函数输入为训练样本数据，开发者需要在该函数中调用算法消费训练样本进行训练。

当然，在不同的训练模式下，该函数使用方法有所不同：

- 单机训练：开发者需要在训练 workflow 中手动调用该函数以进行进一步训练。
- 分布式训练：
 - 该函数作为训练函数会被循环执行，无需开发者手动调用。
 - 但该函数还作为样本发送函数，开发者需要在训练 workflow 中手动调用，以将样本发送至样本池。

```
1 @learn_wrapper
2 def learn(self, list_sample_data):
```

Parameters

参数名	介绍
list_sample_data	list类型，训练样本(SampleData)列表，Learner从样本池按照配置项 <code>train_batch_size</code> 采样一批样本, 作为输入传入 <code>learn()</code> 函数。

配置项

开发目录：`/conf/configure_app.toml`

Learner在每一次执行 `learn()` 函数时，会从样本池采样一批样本作为输入。并按照开发者配置的频次 `dump_model_freq` 保存模型参数文件，模型同步服务将按照配置 `model_file_sync_per_minutes` 将模型参数文件推送至模型池。

Actor中的模型同步服务将按照配置 `model_file_sync_per_minutes`，从模型池获取最新模型参数文件。

相关配置项如下：

```
1 [app]
2 # The time interval for executing the learn() function, configurable to
  # throttle the Learner and balance sample production/consumption.
3 # 执行learn函数进行训练的时间间隔，可通过该配置让Learner休息以调节样本生产消耗比
4 learner_train_sleep_seconds = 0.001
5
6 # Replay buffer configurations
7 # 样本池容量
8 replay_buffer_capacity = 4096
9
10 # The ratio of the sample pool capacity that triggers training
11 # 当样本池中的样本占总容量的比例达到该值时，启动训练
12 preload_ratio = 1.0
13
14 # When new samples are added to the sample pool, the logic for removing old
  # samples: reverb.selectors.Lifo, reverb.selectors.Fifo
15 # 当新样本加入样本池时，旧样本的移除逻辑，可选项: reverb.selectors.Lifo,
  # reverb.selectors.Fifo
16 # reverb.selectors.Lifo: 先进后出(Last In, First Out)
17 # reverb.selectors.Fifo: 先进先出(First In, First Out)
18 reverb_remover = "reverb.selectors.Fifo"
19
20 # The sampling logic of the Learner from the sample pool:
  # reverb.selectors.Fifo, reverb.selectors.Uniform
21 # Learner从样本池中采样的逻辑，可选项: reverb.selectors.Fifo,
  # reverb.selectors.Uniform
22 # reverb.selectors.Uniform: Samples are selected uniformly at random from the
  # replay buffer, with each sample having an equal probability of being chosen.
23 # reverb.selectors.Uniform: 从回放缓冲区中随机均匀地选择样本，每个样本被选中的概率相
  # 同。
24 # reverb.selectors.Fifo: Samples are selected in the order they were added to
  # the replay buffer.
25 # reverb.selectors.Fifo: 按照先进先出从回放缓冲区中选择样本。
26 reverb_sampler = "reverb.selectors.Uniform"
27
28 # Training batch size limit for Learner
29 # Learner训练时样本批处理大小
30 train_batch_size = 2048
31
32 # Model dump frequency (steps)
33 # 训练间隔多少步输出模型参数文件
34 dump_model_freq = 1000
35
36 # The Learner pushes model updates, and the frequency at which Actors fetch
  # the model (in minutes).
37 # Learner推送模型参数文件至模型池，以及Actor从模型池获取模型参数文件的频次（单位：分钟）
```

```

38 model_file_sync_per_minutes = 1
39
40 # he number of model updates pushed per learner iteration, and the maximum
  number of updates each actor can fetch at once (cap: 50).
41 # Learner每次推送模型参数文件，以及Actor每次获取模型参数文件的数量（上限：50）
42 modelpool_max_save_model_count = 1

```

predict

该方法通过调用模型进行预测，通常在训练时调用该方法，依策略的概率分布采样或引入随机概率。

```

1 @predict_wrapper
2 def predict(self, list_obs_data, list_state):
3     return [ActData]

```

Parameters

参数名	介绍
list_obs_data	list类型，观测数据(ObsData)列表
list_state	可选参数，list类型，环境返回的状态数据列表

Return

参数名	介绍
List(ActData)	list类型，开发者定义的动作数据(ActData)列表

exploit

该方法通过调用模型进行预测，通常在评估时调用该方法，选取策略中概率最高的动作或者策略认为最优的动作。

```

1 @exploit_wrapper
2 def exploit(self, observation):

```

Parameters

参数名	介绍
observation	dict类型，环境观测字典，评估工作流程中将原始的环境观测信息作为输入传入 <code>agent.exploit()</code> 。

Return

参数名	介绍
action	list类型，动作列表，环境可以直接使用的动作指令

load_model

智能体通过该接口完成模型参数加载。在上文中提到，Actor会从模型池中获取最新模型参数文件，开发者需要手动调用 `load_model()` 函数，使智能体完成模型参数加载。

```
1 @load_model_wrapper
2 def load_model(self, path=None, id="1"):
3     # when loading the model, you can load multiple files,
4     # and it is important to ensure that each filename matches the one used
5     # during the save_model process.
6     # 加载模型，可以加载多个文件，注意每个文件名需要和save_model时保持一致
7     model_file_path = f"{path}/model.ckpt-{str(id)}.pkl"
8     self.model.load_state_dict(
9         torch.load(model_file_path, map_location=self.device),
```

Parameters

参数名	介绍
path	string类型，加载模型参数文件的路径，开发框架根据使用场景得到相应的路径, 并作为输入传入 <code>load_model</code>
id	string类型，模型参数文件的 id，使用 id 指定加载的模型参数文件

save_model

开发者可以通过该函数保存当前时刻的模型文件及智能体代码包，开发框架会将开发者需要保存的内容打包为zip格式的文件。

当开发者使用腾讯开悟客户端开发时，开发框架会在客户端指定目录下存储该zip文件。
当开发者使用腾讯开悟平台时，开发框架会将该zip文件存储在云端，开发者可以通过平台的训练管理模块查看每一个训练任务的zip文件，即模型。

```
1 @save_model_wrapper
2 def save_model(self, path=None, id="1"):
3     # To save the model, it can consist of multiple files,
4     # and it is important to ensure that each filename includes the
5     # "model.ckpt-id" field.
6     # 保存模型，可以是多个文件，需要确保每个文件名里包括了model.ckpt-id字段
7     model_file_path = f"{path}/model.ckpt-{str(id)}.pkl"
8
9     # Copy the model's state dictionary to the CPU
10    # 将模型的状态字典拷贝到CPU
11    model_state_dict_cpu = {k: v.clone().cpu() for k, v in
12    self.model.state_dict().items()}
13    torch.save(model_state_dict_cpu, model_file_path)
```

Parameters

参数名	介绍
path	string类型，模型文件保存的路径，开发框架根据使用场景得到相应的路径, 并作为输入传入 <code>save_model</code>
id	string类型，模型文件的索引，开发框架获取到模型池中最新模型的索引, 并作为输入传入 <code>save_model</code>

其他功能

加载预训练模型

本框架支持在已有的模型（称为预训练模型）基础上继续训练。预训练模型结构和继续训练的模型结构需保持一致。

- 当使用腾讯开悟平台时，在创建训练任务时，可以在弹窗中选择预训练模型以继续训练。
- 当使用腾讯开悟客户端时，需要将训练好的模型文件（客户端工作空间/ckpt 路径下的pkl类型文件）放入代码包中，并在代码包 `conf/configure_app.toml` 中完成预训练模型相关的配置，配置说明如下：

```

1 # whether to enable the preload model function. If enabled (true), the model
  specified by preload_model_id will be loaded as the initial model in the
  preload_model_dir directory; if disabled (false), no preloading will be
  performed.
2 # 是否启用加载预训练模型功能，若开启(true)，将在preload_model_dir目录下加载由
  preload_model_id指定的模型作为初始模型；若关闭(false)，则不加载预训练模型。
3 preload_model = false
4
5 # The relative path of the preloaded model folder (the variable name
  {agent_name} refers to the agent_algorithm name directory in the code
  package). It is only effective when preload_model=true. When the preload
  model function is enabled, you need to create a new ckpt folder under the
  agent_algorithm name directory in the code package and place the model file
  (.pkl) there.
6 # 预训练模型文件夹相对路径(变量名{agent_name}指代码包中agent_算法名目录)，仅在
  preload_model=true时生效；当开启加载预训练模型功能时，需要在代码包中agent_算法名目录下
  新建ckpt文件夹，将模型文件(.pkl)放置此即可。
7 preload_model_dir = "{agent_name}/ckpt"
8
9 # The identification ID of the preloaded model (here refers to the number of
  model training steps). This ID corresponds to the number of training steps
  recorded in the model file name. It only takes effect when
  preload_model=true.
10 # Note that it is forbidden to modify the original model file name, otherwise
  the model preloading process will fail.
11 # 预训练模型的标识ID（这里指模型训练步数），该ID对应模型文件名中的训练步数记录。仅在
  preload_model=true时生效。
12 # 注意，禁止修改原始模型文件名，否则将导致预训练模型加载失败。
13 preload_model_id = 1000

```

客户端继续训练使用示例

以加载 40052步的DQN模型 为例：

- 解压模型压缩包，在ckpt路径下找到名为 `model.ckpt-40052.pkl` 的模型文件
- 将模型文件放到代码包路径下，例如在 `agent_dqn` 内新建一个 `ckpt` 目录，放入上述模型文件
- 在代码包 `conf/configure_app.toml` 中完成以下配置：

```

1 preload_model = true
2 preload_model_dir = "agent_dqn/ckpt"
3 preload_model_id = 40052

```

完成上述步骤后，Aisrv和Learner将加载预训练模型以继续训练。

加载对手模型功能

在使用腾讯开悟平台时，部分项目支持从平台的模型管理中加载自定义模型。用户可以在 `kaiwu.json` 文件中配置期望用作对手模型的模型 ID。智能体通过 `load_opponent_agent()` 函数完成对手模型的加载。

该功能允许智能体加载指定的网络结构和参数，并根据模型文件中的实现调用 agent 的 `reset`、`predict`、`exploit` 等方法。对手模型作为固定水平的自定义模型，可以在 PVP 任务中用于训练评估，以反映训练过程中模型水平的提高。

以下是 `load_opponent_agent()` 函数示例：

```
1 | @load_opponent_agent_wrapper
2 | def load_opponent_agent(self, id="1"):
3 |     # Framework provides loading opponent agent function, no need to implement
    function content
4 |     # 框架提供的加载对手模型功能，无需实现函数内容
5 |     pass
```

Parameters

参数名	介绍
id	string类型，平台模型管理的 模型id，使用 id 指定加载的模型文件

以下是kaiwu.json的配置示例：

```
1 | {
2 |     "model_pool": [609, 608]
3 | }
```

策略类型选择

本框架支持两种核心的训练范式：**同策略（On-Policy）**与**异策略（Off-Policy）**。此选择是训练流程的基础，将直接影响数据的使用方式、内存占用以及部分超参数的推荐设置。

特性	On-Policy	Off-Policy
数据来源	必须使用由当前正在训练的策略实时与环境交互产生的 新数据 。	可以使用 历史经验数据 ，这些数据可能来自旧版本的策略或不同的探索策略。
数据使用	一次性为主 。数据在用于一次策略更新后通常被丢弃，以确保训练数据与当前策略的一致性。	可重复利用 。数据被存储在“经验回放缓冲区”中，可被多次采样用于训练。
主要优势	训练稳定 ，理论收敛性有保障，行为容易理解。	数据效率高 ，可充分利用每一次交互获得的数据，更适合现实世界中交互成本高的场景。

配置参数详解

在代码包 `conf/configure_app.toml` 中，通过以下参数进行设置：

```
1  # Training paradigm: on-policy or off-policy
2  # 训练时采用on-policy, off-policy
3  algorithm_on_policy_or_off_policy = "on-policy"
4
5  # Model dump frequency (steps)
6  # Interval (in steps) for saving model parameter files.
7  # on_policy: set to 1 for optimal training.
8  # off_policy: recommend 1000 to reduce save frequency.
9  # 训练间隔多少步输出模型参数文件，on_policy时设置为1训练效果最好，off_policy时建议设置
   # 为1000，减少模型保存频率
10 dump_model_freq = 1
```

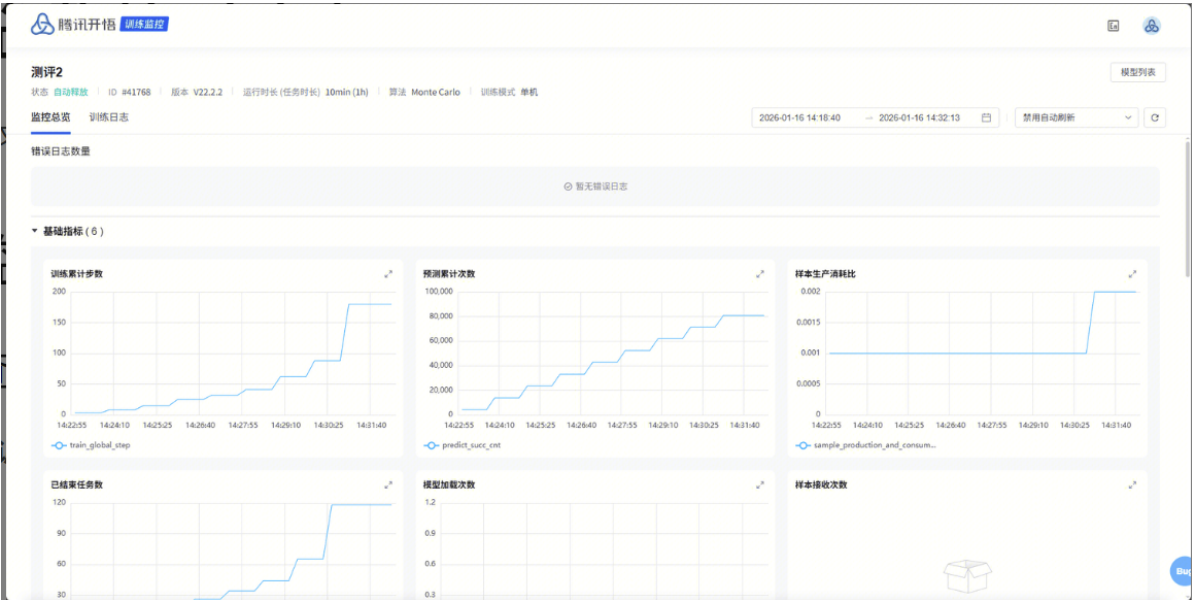
监控与日志

本文介绍如何使用腾讯开悟平台的监控与日志功能，帮助您实时掌握训练状态、快速定位问题。

监控

查看监控面板

在训练管理页面，点击查看监控按钮，即可打开监控面板：



监控面板组成

监控面板包含两个核心模块：

模块	功能
错误日志数量	展示各模块的错误日志统计，点击可查看详情
监控指标图	展示训练过程中的各类数据指标

指标分类

监控指标分为四类：

分类	说明
基础指标 (basic)	训练进度相关的核心数据，如训练步数、预测次数等
硬件指标 (hardware)	资源使用情况，如 CPU、GPU、内存利用率
算法指标 (algorithm)	算法相关数据，不同算法的指标有所不同
环境指标 (env)	环境相关数据，不同环境的指标有所不同

基础指标

面板名称	指标名称	说明
训练累计步数	train_global_step	<code>agent.learn()</code> 的调用次数
预测累计次数	predict_succ_cnt	<code>agent.predict()</code> 的调用次数
模型加载次数	load_model_succ_cnt	<code>agent.load_model()</code> 成功调用的次数
样本接收次数	sample_receive_cnt	接收到的样本总数
已结束任务数	episode_cnt	已完成的 episode 数量
样本生产消耗比	sample_production_and_consumption_ratio	训练消耗样本数 / 采样生产样本数

硬件指标

面板名称	指标名称	说明
CPU 使用率	aisrv_cpu_usage / learner_cpu_usage	分别对应 aisrv 和 learner 进程
GPU 使用率	aisrv_gpu_usage / learner_gpu_usage	分别对应 aisrv 和 learner 进程
GPU 显存使用率	gpu_memory	显存占用百分比
内存使用率	ram_usage	容器内存占用，过高可能导致 OOM

算法指标

不同算法的指标各不相同，详见具体算法文档。

环境指标

不同环境的指标各不相同，详见具体环境文档。

自定义监控面板

平台支持在实验代码中自定义监控指标，配置并上报期望观测的业务数据。

配置面板

在算法配置目录下编辑监控配置文件：

文件路径： `agent_{算法名}/conf/monitor_builder.py`

配置示例：

```
1  def build_monitor():
2      monitor = MonitorConfigBuilder()
3
4      config_dict = (
5          monitor.title("项目名称")
6          .add_group(group_name="算法指标", group_name_en="algorithm")
7          .add_panel(
8              name="累积回报",
9              description="反映智能体能力的指标",
10             type="line",
11         )
12         .add_metric(metrics_name="reward", expr="avg(reward{})")
13         .end_panel()
14         .end_group()
15         .build()
16     )
17     return config_dict
```

上报指标数据

在代码中调用监控上报接口，**代码示例：**

```
1  import os
2  from monitor import monitor
3
4  monitor_data = {
5      "reward": 100.5
6  }
7  monitor.push_data({os.getpid(): monitor_data})
```

配置参数说明

配置项	字段	字段类型	说明	限制条件
项目名称	title	string	监控面板名称（该字段不在监控页面进行展示，不建议修改）	支持中英文、数字、 <code>=</code> 、 <code>+</code> 、 <code>/</code> 、 <code>@</code> 、 <code>#</code> 、 <code>_</code> 、 <code>-</code> 及空格，长度 1~100 字符
面板组	group_name	string	面板组名称	支持中英文、数字、 <code>_</code> 、 <code>-</code> 及空格，长度 1~20 字符
	group_name_en	string	面板组英文标识	支持英文、数字、 <code>_</code> 、 <code>-</code> 及空格，长度 1~50 字符
面板	name	string	面板名称	支持中英文、数字、 <code>_</code> 、 <code>-</code> 及空格，长度 1~20 字符
	name_en	string	面板英文标识	支持英文、数字、 <code>_</code> 、 <code>-</code> 及空格，长度 0~50 字符
	description	string	面板描述信息	支持中英文、数字、标点符号等，长度 0~200 字符
	type	string	图表类型	仅 <code>line</code> （折线图）、 <code>stat</code> （数值图）有效
	unit	string	数值单位，当 <code>type</code> 为 <code>stat</code> 时展示到指标后	仅 <code>stat</code> 类型有效
指标	metrics_name	string	指标显示名称	支持中英文、数字、 <code>_</code> 、 <code>-</code> 、 <code>{}</code> 及空格，长度 1~40 字符
	expr	string	指标查询表达式，支持指标变量（lable）	使用 PromQL 的查询语法即可

规格限制：

- 折线图面板：指标数量限制为 20 个指标
- 数值图面板：指标数量限制为 2 个指标

指标变量说明

如需按维度分组展示数据（如不同对手的胜率），可通过指标变量（label）区分维度，并在配置中使用指标变量定义展示方式。

第一步：上报带 label 的数据

```
1 # 上报玩家1的胜率指标
2 monitor.push_data({os.getpid(): win_rate, "player": "player1"})
3
4 # 上报玩家2的胜率指标
5 monitor.push_data({os.getpid(): win_rate, "player": "player2"})
```

第二步：配置指标变量

```
1 .add_metric(
2     metrics_name="win_rate_{player}",
3     expr="avg(win_rate{ }) by (player)"
4 )
```

说明：

- metrics_name字符串中的 {player} 会被实际的 player1、player2 替换；最终将在同一个面板上显示两条数据线：win_rate_player1 和 win_rate_player2

日志

框架提供了统一的日志服务，帮助您记录和排查训练过程中的问题。

日志格式

字段	说明	示例
time	时间戳	2024-09-18 19:33:04.813469
level	日志级别	INFO / WARNING / ERROR
message	日志内容	kaiwu learner train count is 365676
file	源码文件	on_policy_trainer.py
line	代码行号	769
module	所属模块	learner
process	进程名	on_policy_trainer
function	函数名	train_stat
stack	错误堆栈	仅错误日志包含

注意：

- 不要重写日志系统：**重写后监控面板将无法统计错误日志数量
- 日志流量限制：**框架限制为 60 条/分钟，超出部分将被丢弃